

Task 1 – Test-Driven Development for Even/Odd Number Validator

Task Description

Use AI tools to first generate test cases for a function `is_even(n)` and then implement the function so that it satisfies all generated tests.

Requirements

- Input must be an integer
- Handle zero, negative numbers, and large integers

Example Test Scenarios

`is_even(2)` → True

`is_even(7)` → False

`is_even(0)` → True

`is_even(-4)` → True

`is_even(9)` → False

AI Prompt Used

Generate test cases for a Python function `is_even(n)` where input must be an integer and the function should handle zero, negative numbers, and large integers.

AI Generated Test Cases

Input	Expected Output
<code>is_even(2)</code>	True
<code>is_even(7)</code>	False
<code>is_even(0)</code>	True
<code>is_even(-4)</code>	True
<code>is_even(9)</code>	False
<code>is_even(1000000)</code>	True

Implementation Code

The screenshot shows a Jupyter Notebook titled 'AIAC_8.ipynb' with a dark theme. The interface includes a top menu bar (File, Edit, View, Insert, Runtime, Tools, Help), a command palette, and a sidebar with icons for file management and execution. The main code area is divided into cells. The first cell contains a Python function `is_even(n)` with a docstring, arguments, returns, and raises sections. The second cell contains a `unittest` test class `TestIsEven` with a single test case `test_zero_is_even`. The third cell contains more test cases for negative numbers, large numbers, and non-integer inputs. The fourth cell shows the execution of the tests, resulting in a summary: 'Ran 8 tests in 0.010s' and 'OK'. On the right side, there is a Gemini chat interface with a prompt to generate test cases for the `is_even` function and several suggested actions like 'Show me how to use the is_even function' and 'Explain the is_even function code'.

```
[1] def is_even(n):  
    """  
    Checks if an integer is even.  
  
    Args:  
        n: An integer.  
  
    Returns:  
        True if n is even, False otherwise.  
  
    Raises:  
        TypeError: If n is not an integer.  
    """  
    if not isinstance(n, int):  
        raise TypeError("Input must be an integer.")  
    return n % 2 == 0
```

Now, let's create the test cases using Python's `unittest` module.

```
[2] import unittest  
  
class TestIsEven(unittest.TestCase):  
  
    def test_zero_is_even(self):  
        self.assertTrue(is_even(0), "Test Case Failed: 0 should be even")
```

```
[2] def test_negative_even_number(self):  
    self.assertTrue(is_even(-2), "Test Case Failed: -2 should be even")  
    self.assertTrue(is_even(-100), "Test Case Failed: -100 should be even")  
  
    def test_negative_odd_number(self):  
    self.assertFalse(is_even(-1), "Test Case Failed: -1 should be odd")  
    self.assertFalse(is_even(-99), "Test Case Failed: -99 should be odd")  
  
    def test_large_even_number(self):  
    self.assertTrue(is_even(2**63 - 2), "Test Case Failed: A large even number should be even")  
  
    def test_large_odd_number(self):  
    self.assertFalse(is_even(2**63 - 1), "Test Case Failed: A large odd number should be odd")  
  
    def test_non_integer_input(self):  
    with self.assertRaises(TypeError, msg="Test Case Failed: Non-integer input should raise TypeError"):  
        is_even(3.14)  
    with self.assertRaises(TypeError, msg="Test Case Failed: String input should raise TypeError"):  
        is_even("hello")  
    with self.assertRaises(TypeError, msg="Test Case Failed: None input should raise TypeError"):  
        is_even(None)
```

To run these tests, you can use the following code:

```
[3] if __name__ == '__main__':  
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

```
***  
-----  
Ran 8 tests in 0.010s  
  
OK
```

Code Explanation

The function validates that the input is an integer using type checking. The modulus operator is used to determine whether the number is divisible by two. If divisible, the function returns `True`; otherwise, `False`.

Task 2 – Test-Driven Development for String Case Converter

Task Description

Ask AI to generate test cases for two functions:

- `to_uppercase(text)`
- `to_lowercase(text)`

Requirements

- Handle empty strings
- Handle mixed-case input
- Handle invalid inputs such as numbers or None

Example Test Scenarios

`to_uppercase("ai coding")` → "AI CODING"

`to_lowercase("TEST")` → "test"

`to_uppercase("")` → ""

`to_lowercase(None)` → Error or safe handling

AI Prompt Used

Generate test cases for uppercase and lowercase conversion functions that handle empty strings, mixed-case input, and invalid inputs.

AI Generated Test Cases

Function	Input	Expected Output
<code>to_uppercase("ai coding")</code>	ai coding	AI CODING
<code>to_lowercase("TEST")</code>	test	
<code>to_uppercase("")</code>	""	
<code>to_lowercase("Ai CoDiNg")</code>	ai coding	
<code>to_lowercase(None)</code>	Error	

Implementation Code

AIAC_8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

```
import unittest

class TestStringConversion(unittest.TestCase):

    # Test cases for to_uppercase function
    def test_uppercase_empty_string(self):
        self.assertEqual(to_uppercase(""), "", "Empty string to uppercase should remain empty")

    def test_uppercase_lowercase_string(self):
        self.assertEqual(to_uppercase("hello"), "HELLO", "Lowercase string should be converted to uppercase")

    def test_uppercase_mixed_case_string(self):
        self.assertEqual(to_uppercase("HeLlO WoRlD"), "HELLO WORLD", "Mixed case string should be converted to uppercase")

    def test_uppercase_already_uppercase_string(self):
        self.assertEqual(to_uppercase("PYTHON"), "PYTHON", "Already uppercase string should remain uppercase")

    def test_uppercase_string_with_numbers_and_symbols(self):
        self.assertEqual(to_uppercase("PyThOn123!@#"), "PYTHON123!@#", "String with numbers and symbols should convert letters to uppercase")

    def test_uppercase_invalid_input_int(self):
        with self.assertRaises(TypeError, msg="Integer input should raise TypeError for to_uppercase"):
            to_uppercase(123)

    def test_uppercase_invalid_input_none(self):
        with self.assertRaises(TypeError, msg="None input should raise TypeError for to_uppercase"):
            to_uppercase(None)

    # Test cases for to_lowercase function
    def test_lowercase_empty_string(self):
        self.assertEqual(to_lowercase(""), "", "Empty string to lowercase should remain empty")

    def test_lowercase_uppercase_string(self):
        self.assertEqual(to_lowercase("HELLO"), "hello", "Uppercase string should be converted to lowercase")
```

Gemini

Generate test cases for uppercase and lowercase conversion functions that handle empty strings, mixed case, and invalid inputs such as numbers or None.

Certainly! Here are the Python functions for `to_uppercase` and `to_lowercase`, along with a robust set of `unittest` test cases. These tests cover empty strings, mixed-case strings, and ensure that appropriate `TypeError` exceptions are raised for invalid inputs like numbers or `None`.

Working...

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
def test_lowercase_mixed_case_string(self):
    self.assertEqual(to_lowercase("HeLlO WoRlD"), "hello world", "Mixed case string should be converted to lowercase")

def test_lowercase_already_lowercase_string(self):
    self.assertEqual(to_lowercase("python"), "python", "Already lowercase string should remain lowercase")

def test_lowercase_string_with_numbers_and_symbols(self):
    self.assertEqual(to_lowercase("PyThOn123!@#"), "python123!@#", "String with numbers and symbols should convert letters to lowercase")

def test_lowercase_invalid_input_int(self):
    with self.assertRaises(TypeError, msg="Integer input should raise TypeError for to_lowercase"):
        to_lowercase(123)

def test_lowercase_invalid_input_none(self):
    with self.assertRaises(TypeError, msg="None input should raise TypeError for to_lowercase"):
        to_lowercase(None)
```

To run these tests, you can use the following code:

```
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)

.....
Ran 22 tests in 0.022s

OK
```

To run these tests, you can use the following code:

```
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

```
.....
-----
Ran 8 tests in 0.010s

OK
```

Code Explanation

The functions first check whether the input is a string. If valid, built-in string methods `.upper()` and `.lower()` are used for conversion. Invalid inputs are handled safely through error handling.

Task 3 – Test-Driven Development for List Sum Calculator

Task Description

Use AI to generate test cases for a function `sum_list(numbers)` that calculates the sum of list elements.

Requirements

- Handle empty lists
- Handle negative numbers
- Ignore or safely handle non-numeric values

Example Test Scenarios

`sum_list([1, 2, 3]) → 6`

`sum_list([]) → 0`

`sum_list([-1, 5, -4]) → 0`

`sum_list([2, "a", 3]) → 5`

AI Prompt Used

Generate test cases for a function that sums list elements while handling empty lists, negative numbers, and ignoring non-numeric values.

AI Generated Test Cases

Input	Expected Output
<code>sum_list([1,2,3])</code>	6
<code>sum_list([])</code>	0
<code>sum_list([-1,5,-4])</code>	0
<code>sum_list([2,"a",3])</code>	5

Implementation Code

AIAC_8.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run All

```
[7] def sum_list_elements(data_list):  
    """  
    Sums all numeric elements in a list, ignoring non-numeric values.  
  
    Args:  
        data_list: A list containing various data types.  
  
    Returns:  
        The sum of all numeric elements in the list.  
    """  
    total_sum = 0  
    for item in data_list:  
        if isinstance(item, (int, float)):  
            total_sum += item  
    return total_sum
```

Now, let's create the test cases using Python's unittest module for the `sum_list_elements` function.

```
[8] import unittest  
  
class TestSumListElements(unittest.TestCase):  
  
    def test_empty_list(self):  
        self.assertEqual(sum_list_elements([]), 0, "Test Case Failed: Empty list should sum to 0")  
  
    def test_positive_numbers(self):  
        self.assertEqual(sum_list_elements([1, 2, 3, 4, 5]), 15, "Test Case Failed: Sum of positive numbers incorrect")  
  
    def test_negative_numbers(self):  
        self.assertEqual(sum_list_elements([-1, -2, -3]), -6, "Test Case Failed: Sum of negative numbers incorrect")  
  
    def test_mixed_positive_and_negative_numbers(self):  
        self.assertEqual(sum_list_elements([1, -2, 3, -4, 5]), 3, "Test Case Failed: Sum of mixed numbers incorrect")
```

Gemini

Generate test cases for a function that sums list elements while handling empty lists, negative numbers, and ignoring non-numeric values.
Certainly! Here's the Python function `sum_list_elements` and a set of unittest test cases to verify its behavior. The tests cover empty lists, lists with positive/negative numbers, mixed numeric and non-numeric values, and floating-point numbers, ensuring only valid numeric elements are summed.
Working...

What can I help you build?
+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

```
def test_non_numeric_values_ignored(self):  
    self.assertEqual(sum_list_elements(["a", None, True, {}, []]), 0, "Test Case Failed: Only non-numeric values should sum to 0")  
  
def test_mixed_numeric_and_non_numeric_values(self):  
    self.assertEqual(sum_list_elements([1, "two", 3.5, None, -4, True, 5]), 5.5,  
        "Test Case Failed: Should sum only numeric values, ignoring others")  
  
def test_large_numbers(self):  
    self.assertEqual(sum_list_elements([10**9, 2 * 10**9, -5 * 10**8]), 2.5 * 10**9, "Test Case Failed: Sum of large numbers incorrect")
```

When these tests, you can use the following code:

```
if __name__ == '__main__':  
    unittest.main(argv=['first-arg-is-ignored'], exit=False)  
  
.....F..F.  
FAIL: test_mixed_numeric_and_non_numeric_values (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric_values)  
Traceback (most recent call last):  
  File "/tmp/ipython-input-2308778257.py", line 27, in test_mixed_numeric_and_non_numeric_values  
    self.assertEqual(sum_list_elements([1, "two", 3.5, None, -4, True, 5]), 5.5,  
AssertionError: 6.5 != 5.5 : Test Case Failed: Should sum only numeric values, ignoring others  
  
FAIL: test_non_numeric_values_ignored (__main__.TestSumListElements.test_non_numeric_values_ignored)  
Traceback (most recent call last):  
  File "/tmp/ipython-input-2308778257.py", line 24, in test_non_numeric_values_ignored  
    self.assertEqual(sum_list_elements(["a", None, True, {}, []]), 0, "Test Case Failed: Only non-numeric values should sum to 0")  
AssertionError: 1 != 0 : Test Case Failed: Only non-numeric values should sum to 0  
  
Ran 31 tests in 0.048s
```

Gemini

Generate test cases for a function that sums list elements while handling empty lists, negative numbers, and ignoring non-numeric values.
Certainly! Here's the Python function `sum_list_elements` and a set of unittest test cases to verify its behavior. The tests cover empty lists, lists with positive/negative numbers, mixed numeric and non-numeric values, and floating-point numbers, ensuring only valid numeric elements are summed.
It looks like the `sum_list_elements` function failed two test cases. The issue arises because `True` in Python is a subclass of `int` and evaluates to `1` in numeric operations. Therefore, the function was incorrectly including `True` in the sum when the intention was to ignore all non-numeric values, including booleans. I will fix the
Accept & Run ✓ Accept
Cancel
What can I help you build?
+ Gemini 2.5 Flash
Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

Code Explanation

The function checks whether the input is a list and iterates through its elements. Only numeric values are added to the total, ensuring safe execution even when non-numeric values exist.

Task 4 – Test Cases for Student Result Class

Task Description

Generate test cases for a `StudentResult` class with methods:

- `add_marks(mark)`

- `calculate_average()`
- `get_result()`

Requirements

- Marks must be between 0 and 100
- Average $\geq 40 \rightarrow$ Pass, otherwise Fail

Example Test Scenarios

Marks: [60, 70, 80] $\rightarrow$ Average: 70 $\rightarrow$ Result: Pass

Marks: [30, 35, 40] $\rightarrow$ Average: 35 $\rightarrow$ Result: Fail

Marks: [-10] $\rightarrow$ Error

AI Prompt Used

Generate test cases for a `StudentResult` class where marks must be between 0 and 100 and result depends on average marks.

AI Generated Test Cases

Marks	Average	Result
[60,70,80]	70	Pass
[30,35,40]	35	Fail
[-10]	Error	

Implementation Code

[10]
✓ Os

```
class StudentResult:
    """
    Manages student marks and calculates results.
    """
    def __init__(self):
        self.marks = {}

    def add_marks(self, subject_name, marks):
        """
        Adds marks for a given subject.

        Args:
            subject_name (str): The name of the subject.
            marks (int): The marks obtained, must be between 0 and 100.

        Raises:
            ValueError: If marks are not between 0 and 100.
        """
        if not (0 <= marks <= 100):
            raise ValueError("Marks must be between 0 and 100.")
        self.marks[subject_name] = marks

    def calculate_average(self):
        """
        Calculates the average marks of all subjects.

        Returns:
            float: The average marks. Returns 0.0 if no marks are added.
        """
        if not self.marks:
            return 0.0
        return sum(self.marks.values()) / len(self.marks)

    def get_result(self):
        """
        Returns:
            str: 'Pass' if average marks are 40 or above, 'Fail' otherwise.
        """
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"
```

Gemini

Generate test cases for a StudentResult class with methods add_marks(), calculate_average(), and get_result() where marks must be between 0 and 100.

✦ Certainly! Here's the StudentResult class with add_marks(), calculate_average(), and get_result() methods, incorporating mark validation. Following that, you'll find a set of unittest test cases to ensure its correctness across various scenarios, including invalid mark inputs, average calculation, and result determination.

🔄 Working...

What can I help you build?
+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

11]
✓ Os

```
import unittest

class TestStudentResult(unittest.TestCase):

    def test_add_valid_marks(self):
        student = StudentResult()
        student.add_marks("Math", 85)
        student.add_marks("Science", 90)
        self.assertEqual(student.marks, {"Math": 85, "Science": 90})

    def test_add_marks_boundary_values(self):
        student = StudentResult()
        student.add_marks("Min", 0)
        student.add_marks("Max", 100)
        self.assertEqual(student.marks, {"Min": 0, "Max": 100})

    def test_add_invalid_marks_below_range(self):
        student = StudentResult()
        with self.assertRaisesRegex(ValueError, "Marks must be between 0 and 100."):
            student.add_marks("Math", -5)

    def test_add_invalid_marks_above_range(self):
        student = StudentResult()
        with self.assertRaisesRegex(ValueError, "Marks must be between 0 and 100."):
            student.add_marks("Science", 101)
```

Now, let's create the test cases using Python's `unittest` module for the `StudentResult` class.


```

def test_calculate_average_no_marks(self):
    student = StudentResult()
    self.assertEqual(student.calculate_average(), 0.0)

def test_calculate_average_with_marks(self):
    student = StudentResult()
    student.add_marks("Math", 70)
    student.add_marks("Science", 80)
    self.assertAlmostEqual(student.calculate_average(), 75.0)

def test_calculate_average_with_mixed_marks(self):
    student = StudentResult()
    student.add_marks("Sub1", 50)
    student.add_marks("Sub2", 60)
    student.add_marks("Sub3", 70)
    self.assertAlmostEqual(student.calculate_average(), 60.0)

def test_get_result_pass(self):
    student = StudentResult()
    student.add_marks("Math", 60)
    student.add_marks("Science", 70)
    self.assertEqual(student.get_result(), "Pass")

def test_get_result_fail(self):
    student = StudentResult()
    student.add_marks("Math", 20)
    student.add_marks("Science", 30)
    self.assertEqual(student.get_result(), "Fail")

def test_get_result_boundary_pass(self):
    student = StudentResult()
    student.add_marks("Subject", 40)
    self.assertEqual(student.get_result(), "Pass")

def test_get_result_boundary_fail(self):

```

```

        self.assertEqual(student.get_result(), "Pass")

def test_get_result_boundary_fail(self):
    student = StudentResult()
    student.add_marks("Subject", 39)
    self.assertEqual(student.get_result(), "Fail")

def test_get_result_no_marks(self):
    student = StudentResult()
    self.assertEqual(student.get_result(), "Fail") # 0.0 average is < 40

```

To run these tests, you can use the following code:

```

▶ if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)

*** .....F..F.
=====
FAIL: test_mixed_numeric_and_non_numeric_values (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric_values)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-2300778257.py", line 27, in test_mixed_numeric_and_non_numeric_values
    self.assertEqual(sum_list_elements([1, "two", 3.5, None, -4, True, 5]), 5.5,
AssertionError: 6.5 != 5.5 : Test Case Failed: Should sum only numeric values, ignoring others

=====
FAIL: test_non_numeric_values_ignored (__main__.TestSumListElements.test_non_numeric_values_ignored)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-2300778257.py", line 24, in test_non_numeric_values_ignored
    self.assertEqual(sum_list_elements(["a", None, True, {}, []]), 0, "Test Case Failed: Only non-numeric values should sum to 0")
AssertionError: 1 != 0 : Test Case Failed: Only non-numeric values should sum to 0

-----
Ran 43 tests in 0.041s

```

Code Explanation

The class validates marks before adding them. The average is calculated dynamically, and the result is determined based on whether the average is greater than or equal to 40.

Task 5 – Test-Driven Development for Username Validator

Task Description

Develop a username validation function based on specified rules.

Requirements

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Example Test Scenarios

`is_valid_username("user01")` → True

`is_valid_username("ai")` → False

`is_valid_username("user name")` → False

`is_valid_username("user@123")` → False

AI Prompt Used

Generate test cases for a username validator with minimum length 5, no spaces, and only alphanumeric characters.

AI Generated Test Cases

Input	Expected Output
-------	-----------------

user01	True
--------	------

ai	False
----	-------

user name	False
-----------	-------

user@123	False
----------	-------

Implementation Code

```
import re

def is_valid_username(username):
    """
    Validates a username based on specific criteria:
    - Minimum length of 5 characters.
    - No spaces allowed.
    - Only alphanumeric characters (letters and numbers).

    Args:
        username (str): The username string to validate.

    Returns:
        bool: True if the username is valid, False otherwise.
    """
    if not isinstance(username, str):
        return False
    # Minimum length 5
    if len(username) < 5:
        return False
    # No spaces
    if ' ' in username:
        return False
    # Only alphanumeric characters
    if not re.fullmatch(r'[a-zA-Z0-9]+', username):
        return False
    return True
```

Generate test cases for a username validation function with minimum length 5, no spaces, and only alphanumeric characters.

Okay, here's the Python function `is_valid_username` implementing the validation rules you described. Following that, I've created a set of `unittest` test cases to thoroughly check its behavior, including scenarios for valid usernames, those too short, containing spaces, and having non-alphanumeric characters, as well as handling empty strings and `None` input.

Working...

```
class TestUsernameValidation(unittest.TestCase):

    def test_valid_usernames(self):
        self.assertTrue(is_valid_username("validuser1"), "Test Case Failed: 'validuser1' should be valid")
        self.assertTrue(is_valid_username("user_name"), "Test Case Failed: 'user_name' should be valid") # Note: re.fullmatch will fail if i
        self.assertTrue(is_valid_username("longusername123"), "Test Case Failed: 'longusername123' should be valid")
        self.assertTrue(is_valid_username("UsernamE"), "Test Case Failed: 'UsernamE' should be valid")
        self.assertTrue(is_valid_username("123456"), "Test Case Failed: '123456' should be valid")

    def test_username_too_short(self):
        self.assertFalse(is_valid_username("abc"), "Test Case Failed: 'abc' should be too short")
        self.assertFalse(is_valid_username("abcd"), "Test Case Failed: 'abcd' should be too short")

    def test_username_with_spaces(self):
        self.assertFalse(is_valid_username("user name"), "Test Case Failed: 'user name' should have spaces")
        self.assertFalse(is_valid_username(" username"), "Test Case Failed: ' username' should have leading space")
        self.assertFalse(is_valid_username("username "), "Test Case Failed: 'username ' should have trailing space")

    def test_username_with_non_alphanumeric(self):
        self.assertFalse(is_valid_username("user@name"), "Test Case Failed: 'user@name' should have non-alphanumeric")
        self.assertFalse(is_valid_username("user!123"), "Test Case Failed: 'user!123' should have non-alphanumeric")
        self.assertFalse(is_valid_username("user-name"), "Test Case Failed: 'user-name' should have non-alphanumeric")
        self.assertFalse(is_valid_username("user.name"), "Test Case Failed: 'user.name' should have non-alphanumeric")
        self.assertFalse(is_valid_username("user_name"), "Test Case Failed: 'user_name' should have non-alphanumeric (underscore)")

    def test_empty_username(self):
        self.assertFalse(is_valid_username(""), "Test Case Failed: Empty string should be invalid")

    def test_none_input(self):
        self.assertFalse(is_valid_username(None), "Test Case Failed: None input should be invalid")

    def test_non_string_input(self):
        self.assertFalse(is_valid_username(12345), "Test Case Failed: Integer input should be invalid")
        self.assertFalse(is_valid_username(['a', 'b']), "Test Case Failed: List input should be invalid")
```

To run these tests, you can use the following code:

◆ Gemini

```
▶ if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)

*** .....F..F.....F
=====
FAIL: test_mixed_numeric_and_non_numeric_values (__main__.TestSumListElements.test_mixed_numeric_and_non_numeric_values)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-2300778257.py", line 27, in test_mixed_numeric_and_non_numeric_values
    self.assertEqual(sum_list_elements([1, "two", 3.5, None, -4, True, 5]), 5.5,
AssertionError: 6.5 != 5.5 : Test Case Failed: Should sum only numeric values, ignoring others

=====
FAIL: test_non_numeric_values_ignored (__main__.TestSumListElements.test_non_numeric_values_ignored)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-2300778257.py", line 24, in test_non_numeric_values_ignored
    self.assertEqual(sum_list_elements(["a", None, True, {}, []]), 0, "Test Case Failed: Only non-numeric values should sum to 0")
AssertionError: 1 != 0 : Test Case Failed: Only non-numeric values should sum to 0

=====
FAIL: test_valid_usernames (__main__.TestUsernameValidation.test_valid_usernames)
-----
Traceback (most recent call last):
  File "/tmp/ipython-input-3055724010.py", line 7, in test_valid_usernames
    self.assertTrue(is_valid_username("user_name"), "Test Case Failed: 'user_name' should be valid") # Note: re.fullmatch will fail if it's n
    ~~~~~
AssertionError: False is not true : Test Case Failed: 'user_name' should be valid

-----
Ran 50 tests in 0.046s

FAILED (failures=3)
```

Code Explanation

The function validates the input type, checks length requirements, ensures no spaces are present, and confirms that the username contains only alphanumeric characters.

Conclusion

The lab showed how AI tools can be used to apply the Test Driven Development. The test cases generated by AI have been useful in ensuring that edge cases are discovered and that the program is correct before it is implemented. The procedure enhanced consistency of the code, promoted systematic testing, and emphasized the role of validation and error management in the software development.